

MDC-101 Android：Material Design 元件 (MDC) 基本概念 (Kotlin)

來源：<https://codelabs.developers.google.com/codelabs/mdc-101-kotlin?hl=zh-tw>

步驟數：6

使用 Kotlin 建構包含核心元件的簡易應用程式，藉此瞭解 Android 適用質感設計元件的基本使用概念。

目錄

1. [1. 簡介](#)
2. [2. 設定開發環境](#)
3. [3. 新增文字欄位](#)
4. [4. 新增按鈕](#)
5. [5. 前往下一個片段](#)
6. [6. 大功告成](#)

1. 簡介



開發人員可透過 Material 元件 (MDC) 實作 Material Design。MDC 由 Google 的工程師和 UX 設計師團隊打造，提供數十種美觀實用的 UI 元件，適用於 Android、iOS、網頁和 Flutter。material.io/develop

什麼是 Material Design 和 Android 適用的 Material 元件？

Material Design 是一套系統，可協助您打造大膽且美觀的數位產品。只要在一致的原則和元件下整合樣式、品牌、互動和動態效果，產品團隊就能發揮最大的設計潛力。

對於 Android 應用程式，**Android 適用的 Material 元件 (MDC Android)** 結合了設計和工程，提供元件庫，確保應用程式的設計一致。隨著 Material Design 系統演進，這些元件也會更新，確保實作方式完全一致，並符合 Google 的前端開發標準。MDC 也適用於網頁、iOS 和 Flutter。

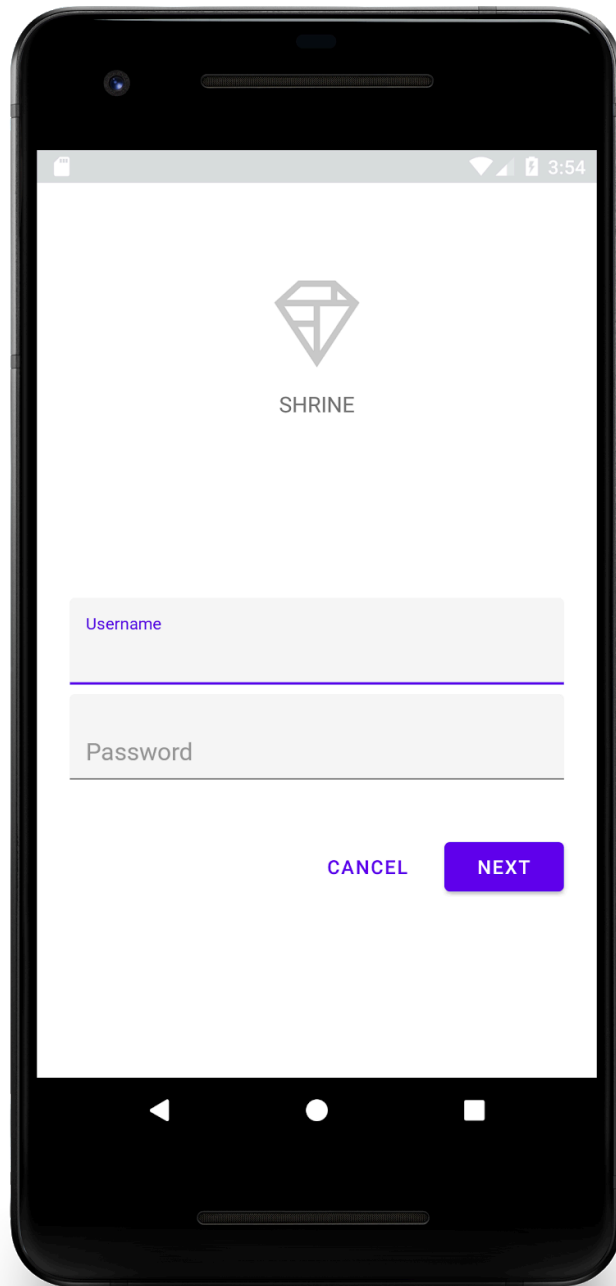
在本程式碼研究室中，您將使用多個 MDC Android 元件建構登入頁面。

建構項目

本程式碼研究室是 4 個程式碼研究室中的第一個，將引導您建構名為「Shrine」的應用程式，這是一個販售服飾和居家用品的電子商務 Android 應用程式。並說明如何使用 MDC Android 自訂元件，反映任何品牌或風格。

在本程式碼研究室中，您將為 Shrine 建構登入頁面，其中包含：

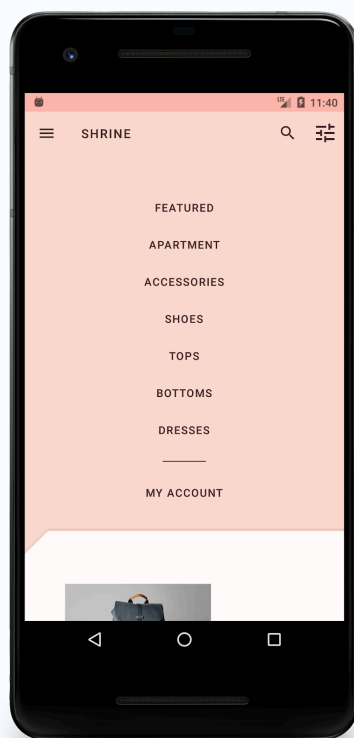
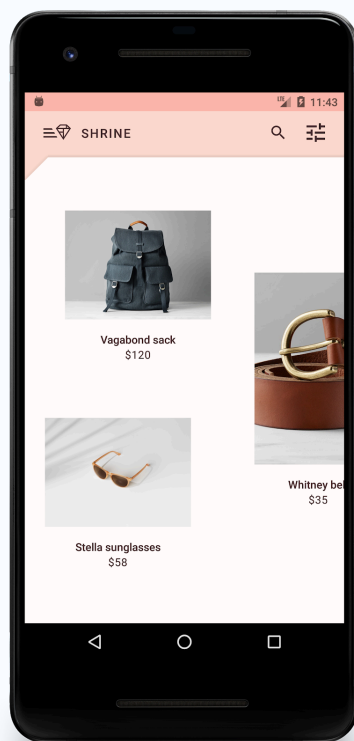
- 兩個文字欄位，一個用於輸入使用者名稱，另一個用於輸入密碼
- 兩個按鈕，分別是「取消」和「下一步」
- 應用程式名稱 (Shrine)
- Shrine 標誌的圖片

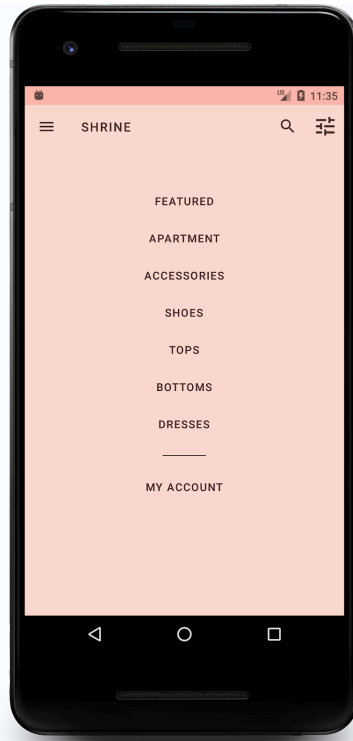


相關程式碼研究室如下：

- [MDC 102：Material Design 結構和版面配置](#)
- [MDC 103：利用顏色、形狀、高度和類型建立 Material Design 主題](#)
- [MDC 104：Material Design 進階元件](#)

完成 MDC 104 後，您將建構出如下所示的 Android 應用程式：





本程式碼研究室中的 MDC Android 元件

- 文字欄位
- 按鈕

在本程式碼研究室中，您將使用 MDC Android 提供的預設元件。您將在 [MDC 103：利用顏色、形狀、高度和類型建立 Material Design 主題](#) 中，瞭解如何自訂這些屬性。

軟硬體需求

- 具備 Android 開發的基本知識
 - **Android Studio** (如果沒有，請[在這裡](#)下載)
 - Android 模擬器或裝置 (可透過 Android Studio 取得)
 - 程式碼範例 (請參閱下一個步驟)
-

2. 設定開發環境

啟動 Android Studio

開啟 **Android Studio** 時，應該會顯示名為「Welcome to Android Studio」的視窗。不過，如果您是第一次啟動 Android Studio，請使用預設值完成 **Android Studio** 設定精靈步驟。下載及安裝必要檔案可能需要幾分鐘的時間，因此您可以讓這項作業在背景執行，同時進行下一節的步驟。

下載程式碼研究室的入門應用程式

範例應用程式位於 `material-components-android-codelabs-101-starter/kotlin` 目錄中。

...或從 GitHub 複製

如要從 GitHub 複製本程式碼研究室，請執行下列指令：

```
git clone https://github.com/material-components/material-components-android-codelabs
cd material-components-android-codelabs/
git checkout 101-starter
```

如需更多說明，請參閱「[從 GitHub 複製存放區](#)」。

在 Android Studio 中載入範例程式碼

1. 設定精靈完成後，系統會顯示「Welcome to Android Studio」視窗，請按一下「Open an existing Android Studio project」。前往安裝程式碼範例的目錄，然後選取「kotlin」->「shrine」(或在電腦上搜尋「shrine」)，開啟 Shipping 專案。
2. 稍待片刻，Android Studio 會建構及同步處理專案，Android Studio 視窗底部的活動指標會顯示進度。
3. 此時，Android Studio 可能會引發一些建構錯誤，因為您缺少 Android SDK 或建構工具，如下所示。按照 Android Studio 中的操作說明安裝/更新這些項目，然後同步處理專案。

 KzoYWC1S7Se7yL8igi1vXF_mbVxAdl2lg5kb7RODrsvPEng0G6U3NK1Qnn0faBBZd2u71yMXioy9tD-7fv3NXvVO4N3EtMMeWDTmqBMMI6egd9R5uXX0T_SKmahbmRor3wZZHX0ByA

新增專案依附元件

專案需要 **MDC Android** 支援程式庫的依附元件。您下載的程式碼範例應該已列出這項依附元件，但建議您執行下列步驟確認。

1. 前往 `app` 模組的 `build.gradle` 檔案，確認 `dependencies` 區塊包含 MDC Android 的依附元件：

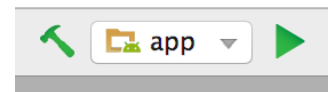
```
api 'com.google.android.material:material:1.1.0-alpha06'
```

2. (選用) 視需要編輯 `build.gradle` 檔案，加入下列依附元件並同步處理專案。

```
dependencies {
    api 'com.google.android.material:material:1.1.0-alpha06'
    implementation 'androidx.legacy:legacy-support-v4:1.0.0'
    implementation 'com.android.volley:volley:1.1.1'
    implementation 'com.google.code.gson:gson:2.8.5'
    implementation "org.jetbrains.kotlin:kotlin-stdlib-jdk7:1.3.21"
    testImplementation 'junit:junit:4.12'
    androidTestImplementation 'androidx.test:core:1.1.0'
    androidTestImplementation 'androidx.test.ext:junit:1.1.0'
    androidTestImplementation 'androidx.test:runner:1.2.0-alpha05'
    androidTestImplementation 'androidx.test.espresso:espresso-core:3.2.0-alpha05'
}
```

執行範例應用程式

1. 確認「Run」/「Play」按鈕左側的建構設定為 `app`。
2. 按下綠色的「Run / Play」按鈕，即可建構及執行應用程式。
3. 在「Select Deployment Target」視窗中，如果可用裝置清單中已有 **Android** 裝置，請跳至步驟 8。如果沒有，請按一下「Create New Virtual Device」。
4. 在「Select Hardware」畫面中，選取手機裝置 (例如 **Pixel 2**)，然後按一下「Next」。
5. 在「系統映像檔」畫面中，選取最新的 **Android** 版本，最好是最高的 API 級別。如果尚未安裝，請按一下顯示的「下載」連結，然後完成下載。
6. 點選 [下一步]。
7. 在 **Android 虛擬裝置 (AVD)** 畫面中，保留預設設定並點選「Finish」。
8. 從部署目標對話方塊中選取 **Android** 裝置。
9. 按一下 [確定]。
10. Android Studio 會建構及部署應用程式，並在目標裝置上自動開啟。



如果無法新增依附元件或順利執行應用程式，請停止並排解開發人員環境的問題。

如果在建構時收到 `INSTALL_FAILED_INVALID_APK` 錯誤，請嘗試在 Android Studio 中依序選取「Build」(建構) -> 「Clean Project」(清除專案)。

太棒了，模擬器應會執行 Shrine 登入頁面的範例程式碼。您應該會看到「Shrine」名稱，以及正下方的 Shrine 標誌。



我們來看看程式碼。我們在程式碼範例中提供簡單的 `Fragment` 導覽架構，可顯示片段並在片段之間導覽。

開啟 `shrine -> app -> src -> main -> java -> com.google.codelabs.mdc.kotlin.shrine` 目錄中的 `MainActivity.kt`。其中應包含下列項目：

注意：如要進一步瞭解 Navigation，以及如何使用 Android Studio 建立及維護導覽路徑，請參閱「[Kotlin Android 基礎知識 03.2：定義導覽路徑](#)」練習。

MainActivity.kt

```

package com.google.codelabs.mdc.kotlin.shrine

import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity
import androidx.fragment.app.Fragment

class MainActivity : AppCompatActivity(), NavigationHost {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.shr_main_activity)

        if (savedInstanceState == null) {
            supportFragmentManager
                .beginTransaction()
                .add(R.id.container, LoginFragment())
                .commit()
        }
    }

    override fun navigateTo(fragment: Fragment, addToBackStack: Boolean) {
        val transaction = supportFragmentManager
            .beginTransaction()
            .replace(R.id.container, fragment)

        if (addToBackStack) {
            transaction.addToBackStack(null)
        }

        transaction.commit()
    }
}

```

這項活動會顯示 `shr_main_activity.xml` 中定義的 `R.layout.shr_main_activity` 版面配置檔案。

您可以看到 `onCreate()`，`MainActivity.kt` 會啟動 `Fragment` 交易，以便顯示 `LoginFragment`。在本程式碼研究室中，我們將修改 `LoginFragment`。活動也會實作 `NavigationHost` 中定義的 `navigateTo(Fragment)` 方法，讓任何片段都能導覽至其他片段。

在活動檔案中按一下 **Command 鍵 + 點選** (或按一下 **Control 鍵 + 點選**) `shr_main_activity`，即可開啟版面配置檔案，或在 `app -> res -> layout -> shr_main_activity.xml` 中前往版面配置檔案。

注意：如果開啟 `shr_main_activity.xml` 時看到的是應用程式預覽畫面，而非 XML 檔案，請從面板左下方的「設計」分頁切換至「文字」分頁。在本程式碼研究室中，我們將全程使用「Text」分頁。

shr_main_activity.xml

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/container"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity"/>
```

這裡會看到簡單的 `<FrameLayout>`，做為活動顯示任何片段的容器。

接著開啟 `LoginFragment.kt`。

LoginFragment.kt

```
package com.google.codelabs.mdc.kotlin.shrine

import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.fragment.app.Fragment

class LoginFragment : Fragment() {

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?, savedInstanceState: Bundle?):
    View? {
        // Inflate the layout for this fragment
        val view = inflater.inflate(R.layout.shr_login_fragment, container, false)

        return view
    }
}
```

`LoginFragment` 會加載 `shr_login_fragment` 版面配置檔案，並顯示在 `onCreateView()` 中。

現在來看看 `shr_login_fragment.xml` 版面配置檔案，瞭解登入頁面的外觀。

shr_login_fragment.xml

```

<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/loginPageBackgroundColor"
    tools:context=".LoginFragment">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:clipChildren="false"
        android:clipToPadding="false"
        android:orientation="vertical"
        android:padding="24dp"
        android:paddingTop="16dp">

        <ImageView
            android:layout_width="64dp"
            android:layout_height="64dp"
            android:layout_gravity="center_horizontal"
            android:layout_marginTop="48dp"
            android:layout_marginBottom="16dp"
            app:srcCompat="@drawable/shr_logo" />

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_gravity="center_horizontal"
            android:layout_marginBottom="132dp"
            android:text="@string/shr_app_name"
            android:textAllCaps="true"
            android:textSize="16sp" />

    </LinearLayout>
</ScrollView>

```

這裡可以看到頂端有 `<ImageView>` 的 `<LinearLayout>`，代表 Shrine 標誌。

接著是 `<TextView>` 標記，代表標誌下方的 Shrine 標籤。這個標籤的文字是名為 `@string/shr_app_name` 的字串資源。如果按住 **Command** 鍵並點選 (或按住 **Control** 鍵並點選) 字串資源名稱，或是開啟 `app -> res -> values -> strings.xml`，您會看到定義字串資源的 `strings.xml` 檔案。日後新增更多字串資源時，也會在此定義。這個檔案中的每個資源都應有 `shr_` 前置字串，表示這些資源屬於 Shrine 應用程式。

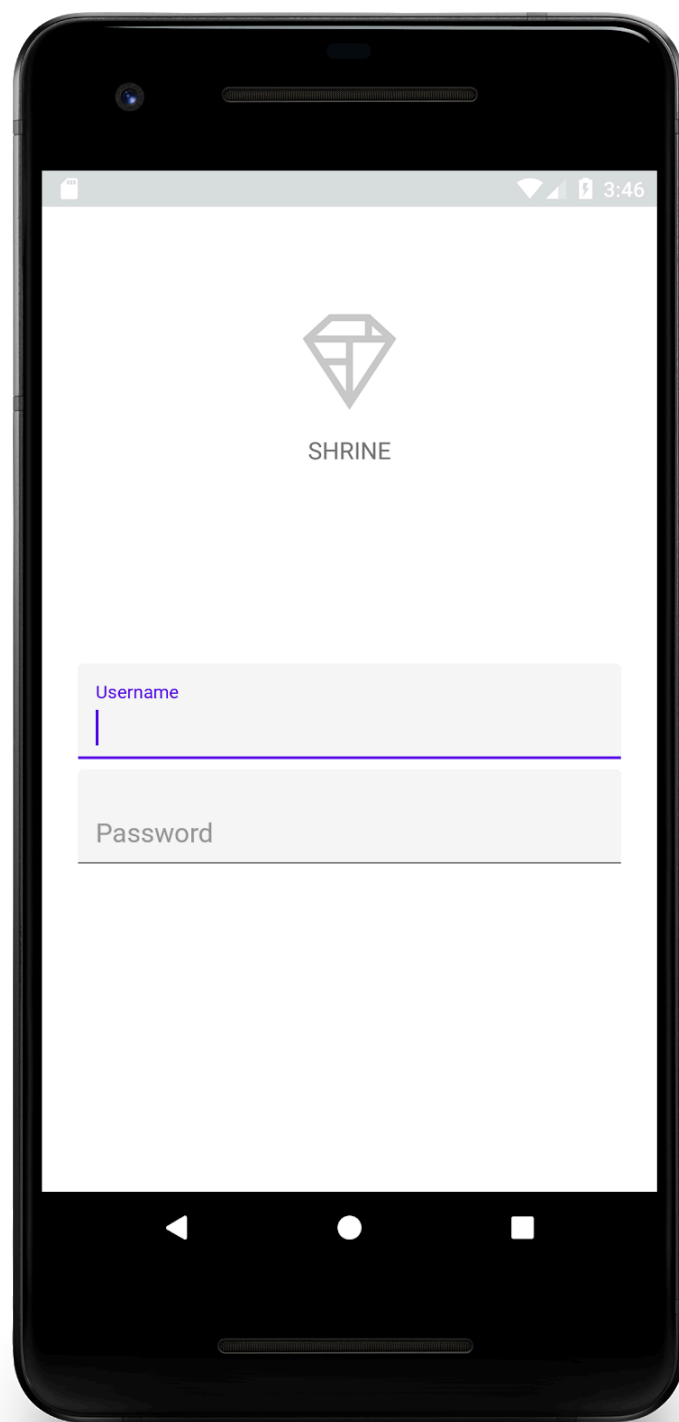
您已熟悉範例程式碼，接下來要實作第一個元件。

3. 新增文字欄位

首先，我們會在登入頁面中加入兩個文字欄位，供使用者輸入使用者名稱和密碼。我們會使用 MDC 文字欄位元件，其中內建可顯示浮動標籤和錯誤訊息的功能。

我們大幅強化了 Material Design 文字欄位，提供更多自訂選項來支援您的品牌，並根據廣泛的使用者體驗 (UX) 研究進行改良，提升可用性和視覺設計。

如要進一步瞭解 Material 指南中的新文字欄位，請參閱[這篇文章](#)。



新增 XML

在 `shr_login_fragment.xml` 中，於「SHRINE」標籤 `<TextView>` 下方的 `<LinearLayout>` 內，新增兩個含有子項 `TextInputEditText` 的 `TextInputLayout` 元素：

shr_login_fragment.xml

```
<com.google.android.material.textfield.TextInputLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_margin="4dp"
    android:hint="@string/shr_hint_username">

    <com.google.android.material.textfield.TextInputEditText
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />
</com.google.android.material.textfield.TextInputLayout>

<com.google.android.material.textfield.TextInputLayout
    android:id="@+id/password_text_input"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_margin="4dp"
    android:hint="@string/shr_hint_password">

    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/password_edit_text"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />
</com.google.android.material.textfield.TextInputLayout>
```

提示：如要在 Android Studio 中快速開啟檔案，請按兩下 **Shift** 鍵，叫出浮動搜尋列，然後輸入要開啟的檔案名稱。

上述程式碼片段代表兩個文字欄位，每個欄位都包含 `<TextInputLayout>` 元素和 `<TextInputEditText>` 子項。每個文字欄位的提示文字都會在 `android:hint` 屬性中指定。

我們為文字欄位新增了兩個字串資源：`@string/shr_hint_username` 和 `@string/shr_hint_password`。開啟 `strings.xml` 即可查看這些字串資源。

strings.xml

```
<string name="shr_hint_username">Username</string>
<string name="shr_hint_password">Password</string>
```

新增輸入內容驗證

`TextInputLayout` 元件內建錯誤意見回饋功能。

如要顯示錯誤意見回饋，請對 `shr_login_fragment.xml` 進行下列變更：

- 在「密碼」 `TextInputLayout` 元素中，將 `app:errorEnabled` 屬性設為 `true`。這樣一來，文字欄位下方的錯誤訊息就會有額外邊框間距。
- 在「密碼」 `TextInputEditText` 元素中，將 `android:inputType` 屬性設為「`textPassword`」。這樣一來，密碼欄位中的輸入文字就會隱藏。

完成這些變更後，`shr_login_fragment.xml` 中的文字欄位應如下所示：

shr_login_fragment.xml

```
<com.google.android.material.textfield.TextInputLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_margin="4dp"
    android:hint="@string/shr_hint_username">

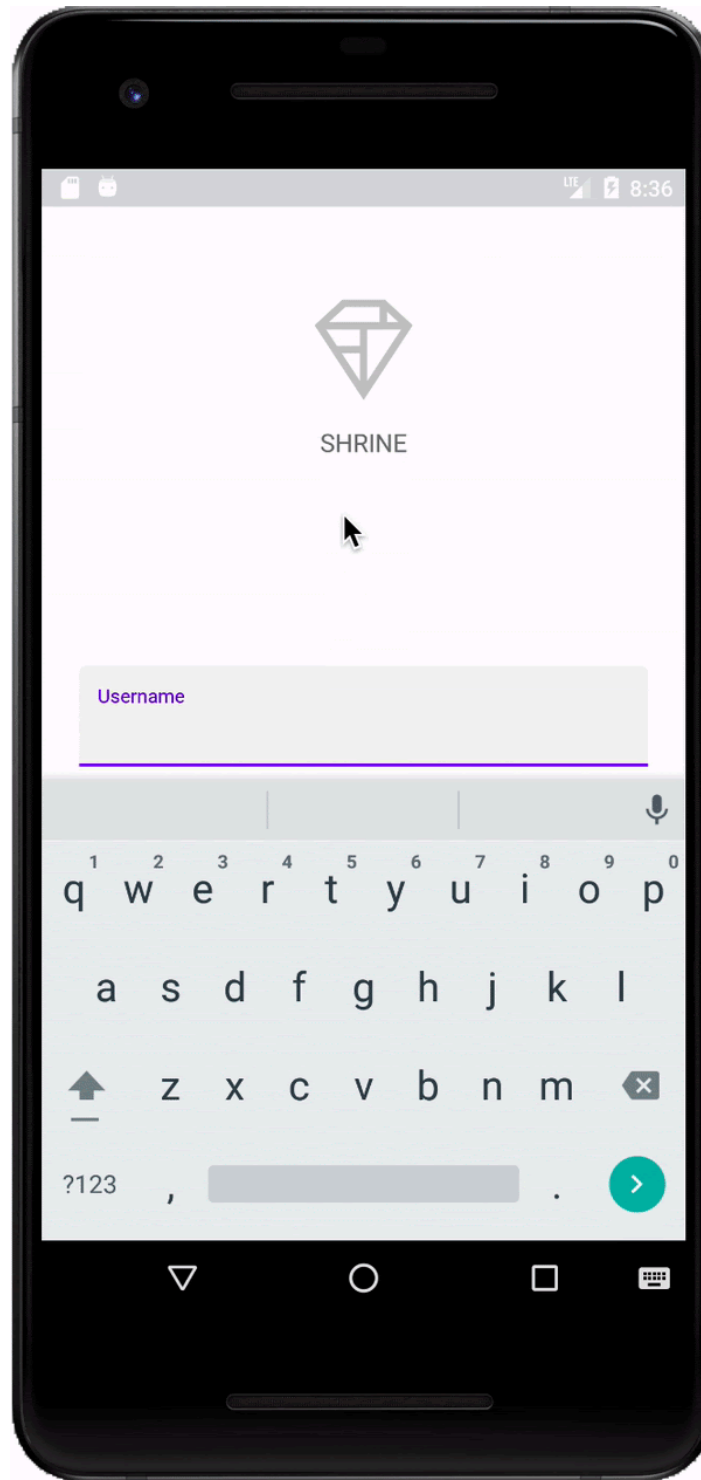
    <com.google.android.material.textfield.TextInputEditText
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />
</com.google.android.material.textfield.TextInputLayout>

<com.google.android.material.textfield.TextInputLayout
    android:id="@+id/password_text_input"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_margin="4dp"
    android:hint="@string/shr_hint_password"
    app:errorEnabled="true">

    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/password_edit_text"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:inputType="textPassword" />
</com.google.android.material.textfield.TextInputLayout>
```

現在請嘗試執行應用程式，您應該會看到「使用者名稱」和「密碼」的兩個文字欄位！

請參閱浮動標籤動畫：



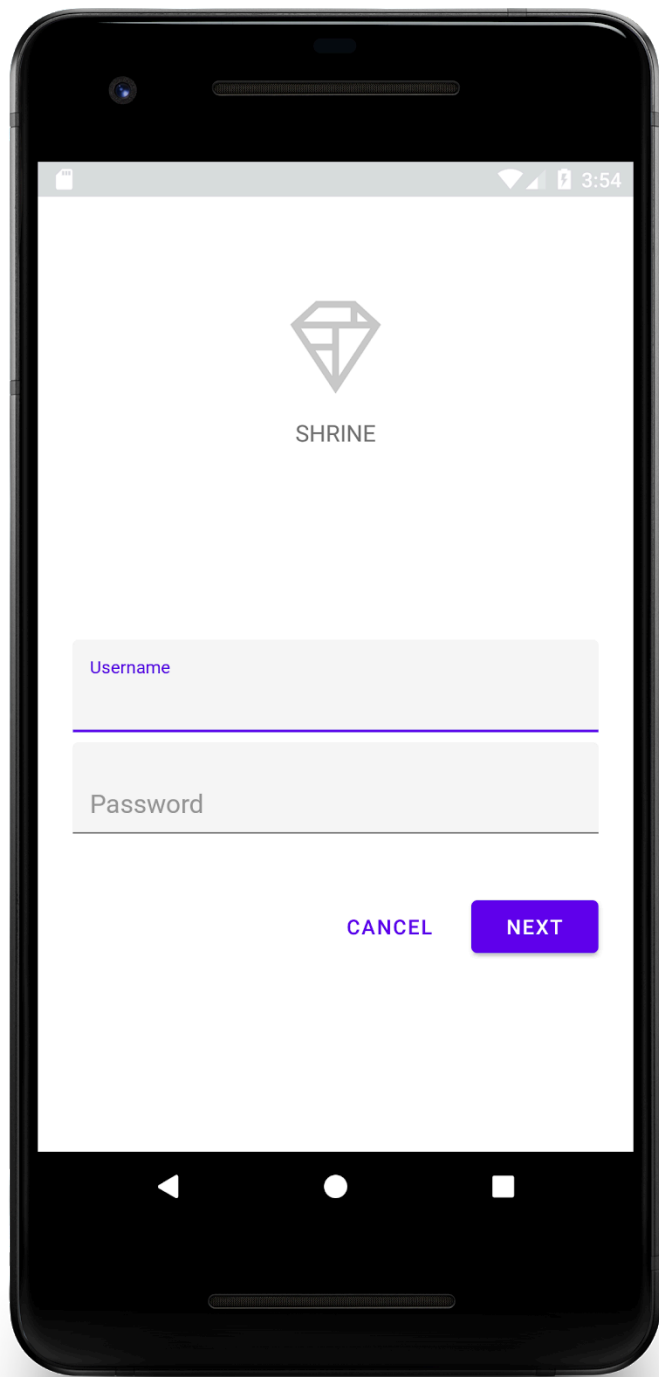
MDC 文字欄位功能包括：

- 顯示內建錯誤回饋
- 使用 `app:passwordToggleEnabled` 支援密碼顯示設定切換鈕
- 使用 `app:helperText` 提供內建輔助文字功能
- 使用 `app:counterEnabled` 和 `app:counterMaxLength` 顯示字元總數和字元數上限

步驟 4 · 約 6 分鐘

4. 新增按鈕

接著，我們會在登入頁面中新增兩個按鈕：「取消」和「下一步」。我們將使用 MDC 按鈕元件，內建標誌性的 Material Design 墨水漣漪效果。



新增 XML

在 `shr_login_fragment.xml` 中，將 `<RelativeLayout>` 新增至 `<LinearLayout>`，位於 `TextInputLayout` 元素下方。接著，在 `<RelativeLayout>` 中新增兩個 `<MaterialButton>` 元素。

產生的 XML 檔案應如下所示：

[shr_login_fragment.xml](#)

```
<RelativeLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content">

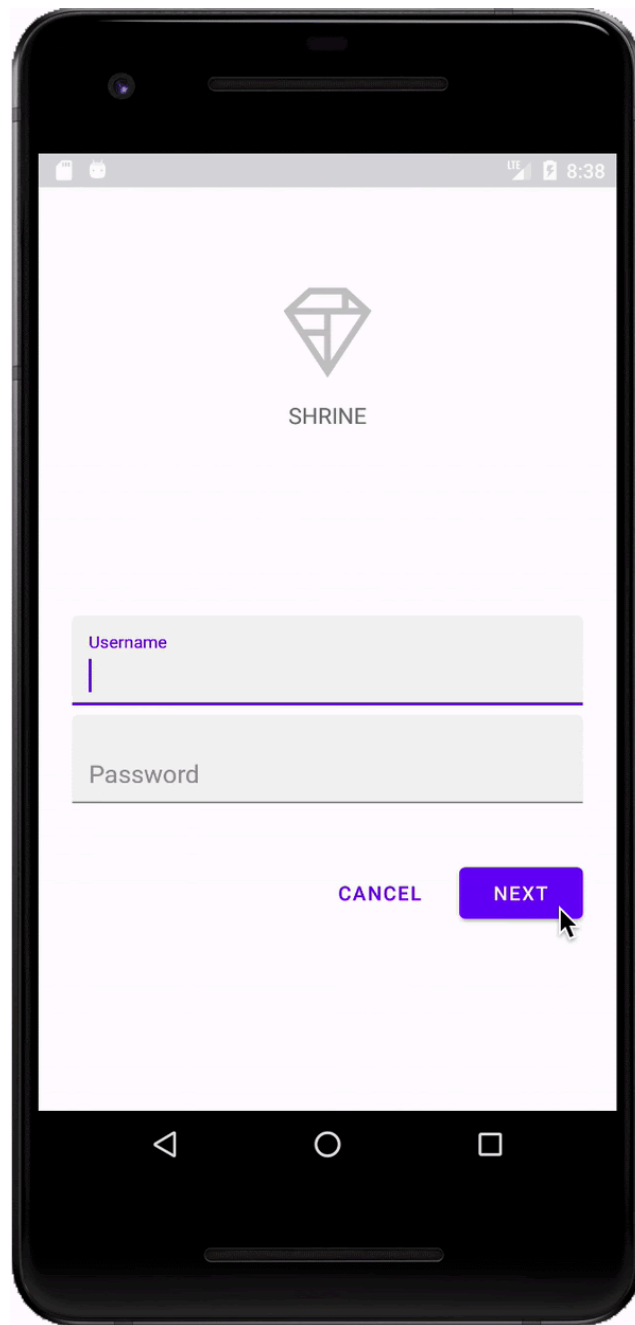
    <com.google.android.material.button.MaterialButton
        android:id="@+id/next_button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_alignParentEnd="true"
        android:layout_alignParentRight="true"
        android:text="@string/shr_button_next" />

    <com.google.android.material.button.MaterialButton
        android:id="@+id/cancel_button"
        style="@style/Widget.MaterialComponents.Button.TextButton"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginEnd="12dp"
        android:layout_marginRight="12dp"
        android:layout_toStartOf="@id/next_button"
        android:layout_toLeftOf="@id/next_button"
        android:text="@string/shr_button_cancel" />

</RelativeLayout>
```

注意： `MaterialButton` 元件的預設樣式為彩色高架按鈕。不過，我們希望這個「取消」按鈕是不會凸起、沒有填滿的按鈕。為達成這個目的，我們會套用 `style` 屬性：`@style/Widget.MaterialComponents.Button.TextButton`。

大功告成！執行應用程式時，輕觸每個按鈕都會顯示墨水漣漪。



MDC 按鈕功能包括：

- 預設內建觸控回饋 (稱為 MDC Ripple)
- 預設海拔高度
- 可自訂的圓角半徑和筆觸

5. 前往下一個片段

最後，我們會在 `LoginFragment.kt` 中新增一些 Kotlin 程式碼，將「NEXT」按鈕連結至另一個片段。

在 `onCreateView()` 下方的 `LoginFragment.kt` 中新增私有布林值 `isPasswordValid` 方法，並加入判斷密碼是否有效的邏輯。就本示範而言，我們只會確保密碼長度至少為 8 個字元：

LoginFragment.kt

```
private fun isPasswordValid(text: Editable?): Boolean {  
    return text != null && text.length >= 8  
}
```

接著，將點擊事件監聽器新增至「Next」按鈕，根據我們剛才建立的 `isPasswordValid()` 方法設定及清除錯誤。在 `onCreateView()` 中，這個點擊事件監聽器應放在膨脹器行和 `return view` 行之間。

現在，請在密碼 `TextInputEditText` 中新增按鍵事件監聽器，監聽會清除錯誤的按鍵事件。這個接聽程式也應使用 `isPasswordValid()` 檢查密碼是否有效。您可以在 `onCreateView()` 中，直接在點擊事件監聽器下方新增這項功能。

您的 `onCreateView()` 方法現在應如下所示：

LoginFragment.kt

```

override fun onCreateView(
    inflater: LayoutInflater, container: ViewGroup?, savedInstanceState: Bundle?):
View? {
    // Inflate the layout for this fragment.
    val view = inflater.inflate(R.layout.shr_login_fragment, container, false)

    // Set an error if the password is less than 8 characters.
    view.next_button.setOnClickListener({
        if (!isPasswordValid(password_edit_text.text!!)) {
            password_text_input.error = getString(R.string.shr_error_password)
        } else {
            // Clear the error.
            password_text_input.error = null
        }
    })

    // Clear the error once more than 8 characters are typed.
    view.password_edit_text.setOnKeyListener({ _, _, _ ->
        if (isPasswordValid(password_edit_text.text!!)) {
            // Clear the error.
            password_text_input.error = null
        }
        false
    })

    return view
}

```

注意：如果 Android Studio 在編寫這段程式碼時發生「Cannot resolve symbol」(無法解析符號) 錯誤，請按 **Alt + Enter** 鍵 (或 Mac 上的 **Option + Return** 鍵) 匯入缺少的符號。

現在，我們可以導覽至另一個片段。在 `onCreateView()` 中，更新 `OnClickListener`，在錯誤驗證成功時導覽至另一個片段。您的 `clickListener` 程式碼現在應如下所示：

LoginFragment.kt

```

// Set an error if the password is less than 8 characters.
view.next_button.setOnClickListener({
    if (!isPasswordValid(password_edit_text.text!!)) {
        password_text_input.error = getString(R.string.shr_error_password)
    } else {
        // Clear the error.
        password_text_input.error = null
        // Navigate to the next Fragment.
        (activity as NavigationHost).navigateTo(ProductGridFragment(), false)
    }
})

```

我們在點擊事件監聽器的 `else` 案例中新增了 `( activity as NavigationHost).navigateTo(ProductGridFragment(), false )` 行。這行程式碼會從 `MainActivity` 呼叫 `navigateTo()` 方法，前往新的片段 `ProductGridFragment`。目前這個頁面是空白的，您會在 MDC-102 中處理這個頁面。

`navigateTo()` 中的 `false` 參數會告知 Activity 不要將目前的 Fragment 新增至返回堆疊，因此使用者無法使用返回鍵返回登入畫面。

接著建構應用程式，然後按下「下一步」按鈕。

你成功了！這個畫面將是下一個程式碼研究室的起點，您會在 MDC-102 中處理這個畫面。

6. 大功告成

您使用基本的 XML 標記和約 30 行 Kotlin 程式碼，透過 Android 適用的 Material Design 元件庫，建立符合 Material Design 指南的精美登入頁面，且在所有裝置上看起來和運作方式都一致。

如要取得完成的 MDC 101 應用程式，請下載[完成的程式碼研究室](#)，或使用 Git 查看 101-complete：`git checkout 101-complete`。

如要測試您製作的頁面版本是否與完成版相同，請在 Android Studio 中開啟該目錄的 **shrine** 專案，然後執行應用程式。

後續步驟

文字欄位和按鈕是 MDC Android 程式庫中的兩個核心元件，但還有許多其他元件！您可以探索 [MDC Android 中的其餘元件](#)。你也可以前往 [MDC 102：Material Design 結構和版面配置](#)，瞭解頂端應用程式列、資訊卡檢視畫面和格線版面配置。感謝您試用 Material Components。希望您喜歡這個程式碼研究室！
